ARE SELF-EVOLVING AGENT SKILLS LEARNING SKILLS?

Anonymous Submission

ABSTRACT

Self-evolving agents store experience in persistent skills, but a higher task score leaves open what useful behavior has become reusable. We diagnose completed SkillEvolver, SPARK, and CoEvoSkills artifacts through repeated paired evaluations and controlled task variations. Our analysis separates recurring aggregate utility from agreement on individual tasks, and decomposes the net gain into improvements and losses. We further distinguish gains on zero-reward tasks from interference with full-reward executions. All three methods improve source-task means in both runs, while task changes expose different scopes of utility. SkillEvolver sustains positive gains across the tested changes by combining more frequent improvements with fewer losses of existing success. These results characterize skill learning through the persistence and distribution of useful behavior beyond the source task.

***Index Terms*—** Agent skills, self-evolution, diagnostic evaluation, skill reuse, generalization

1. INTRODUCTION

Persistent agent skills offer a route to improvement without updating model parameters. Instructions, scripts, and supporting references make execution experience available to later runs. SkillEvolver, SPARK, and CoEvoSkills construct these artifacts through deployment-grounded revision, verified-trajectory distillation, and co-evolutionary verification [1–3]. Their outputs motivate the question in our title: when a system produces a higher-scoring skill, what useful behavior has become persistent?

The location of that improvement matters. A skill can preserve a decision rule that remains useful on new inputs, a choice tailored to the original instance, or a check that a later executor fails to apply. These artifacts may all help on the source task, yet lead to different outcomes when the task changes. The final score also combines artifact construction, execution, and verification. Understanding skill learning therefore requires following the stored guidance into the behavior it supports.

We ask two connected questions. **Does the benefit persist, and under which task changes?** Repeated source-task evaluations test whether paired utility recurs, while frozen-skill variants expose the demands that the same guidance can accommodate. **Which task-level changes produce the apparent improvement?** A positive mean may reflect widespread gains or a small balance between large benefits and losses. Identifying these patterns distinguishes the creation of useful behavior from interference with behavior the executor already performs successfully.

We examine three construction mechanisms through two standard-task runs per method and controlled changes to input representation, task instances, and coupled requirements. Each skill condition is paired with a no-skill execution of the same task. We then match task identities across runs, separate the positive and negative contributions to variant utility, and examine score transitions from the two baseline endpoints. The completed skill in use is the common object of diagnosis.

All three methods yield positive mean source-task gains in both runs, but this agreement does not persist under task variation. SkillEvolver remains positive across all three change types. SPARK's benefit depends on the change type, and CoEvoSkills has negative mean gains in all three. The clearest separation occurs when the instance is replaced. The decomposition explains how these means arise: all three methods produce substantial gains on some variants, while their frequency of improvement and cost to existing successes differ.

Our contribution is a diagnosis of the evidence behind observed skill gains. Repeated evaluations measure agreement across runs; controlled variants locate the scope of utility; gain–loss and baseline-conditioned analyses distinguish added benefits from disruption of existing success.

2. PROBLEM FORMULATION

2.1. Skill Revision and Evaluation

At revision round r, an optimizer updates the current persistent skill artifact:

$$s_{r+1} = O(s_r, \tau_r, f_r), \tag{1}$$

where s_r is the current skill, τ_r is an execution trace, and f_r is feedback from execution and evaluation. The operator O denotes the method's complete update pathway, including its native merge, validation, selection, or adoption mechanism. A generated candidate that is rejected or never adopted is not s_{r+1}.

The agent then executes task x with the frozen skill:

$$o_r = A(x, s_r, z_r), \tag{2}$$

where z_r represents model sampling, tool use, and environment state, and o_r is the resulting output artifact or state change. A verifier maps the output to a score:

$$y_r = V(x, o_r). \tag{3}$$

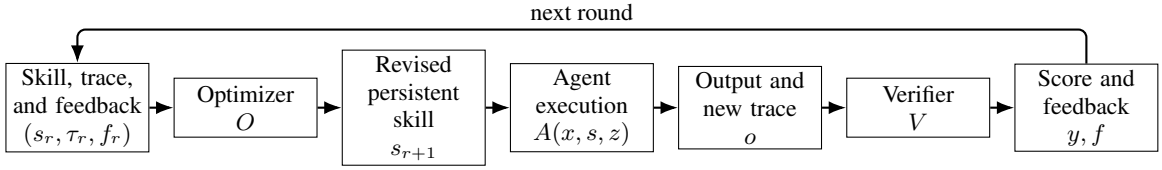


Fig. 1. The skill-evolution lifecycle. Construction produces a persistent artifact; deployment turns its guidance into behavior; verification assigns the observed score. Our evaluation measures completed-artifact utility through paired outcomes, repeated runs, and frozen-skill task variation.

2.2. From Score Gains to Skill Behavior

The score $V(x, A(x, s, z))$ records the outcome of a skill used by an executor and assessed by a verifier. Comparing a completed skill with no skill measures its utility at deployment. Comparing the adopted final skill with its actual parent addresses the improvement associated with revision. The experiments here follow the first contrast across repeated runs and changed tasks.

For each source task x, we freeze its completed skill s_x and evaluate it on a variant $T_k(x)$ alongside a no-skill execution. The skill remains unchanged, while the executor can reinterpret the input, recompute a solution, and adapt its actions. A positive paired difference on the variant indicates that the stored guidance remains useful under that change. The variation types distinguish the decisions that can be reused from those that the executor must reconsider.

To interpret the observed utility, we examine its distribution over tasks. Agreement between runs describes whether the same tasks benefit repeatedly. Separating positive and negative score changes reveals whether a small mean reflects small effects or cancellation. Conditioning on the no-skill score distinguishes help on zero-reward tasks from harm to full-reward executions. These measurements retain the paired task as the unit of analysis while answering different questions about the behavior associated with the skill.

3. EXPERIMENTS

We first test whether completed skills provide a recurring benefit on their source tasks. We then change the task while keeping its skill frozen to determine which demands that benefit survives. The paired no-skill execution provides the reference for both comparisons.

3.1. Experimental Setup

We evaluate SkillEvolver [1], SPARK [2], and CoEvoSkills [3] on SkillsBench [4], with two runs per method. Each run pairs the method's skill condition with a no-skill execution of the same task. Both runs of each method use the same task set: 81 tasks for SkillEvolver, 84 for SPARK, and 85 for CoEvoSkills.

We report mean verifier reward on a 0–100 scale and the mean within-task difference between the skill and no-skill conditions. For a run with N tasks, the paired gain is

$$\Delta = \frac{100}{N} \sum_{i=1}^{N} \left(r_i^{\text{skill}} - r_i^{\text{no-skill}} \right). \tag{4}$$

We also count tasks with positive, negative, and zero differences.

3.2. Positive Source-Task Utility in Both Runs

All three methods have positive mean paired gains in both runs (Table 1). SkillEvolver gains 25.61 and 25.02 points, CoEvoSkills gains 10.24 and 9.76, and SPARK gains 1.84 and 6.45. SkillEvolver and CoEvoSkills retain similar gains despite substantial changes in their no-skill scores. The paired comparison reveals recurring utility that absolute skill-condition scores alone would obscure.

The task-level results also differ. SkillEvolver has at least four times as many improvements as regressions in each run; CoEvoSkills improves 17 tasks in each run, with nine and ten regressions. SPARK's first run is nearly balanced, while its second has 13 improvements and five regressions. The gain therefore reflects both the size and frequency of task-level changes. The frozen-skill variants test whether the useful guidance behind these source-task outcomes survives a different task demand.

3.3. Task Variation Tests What Remains Useful

We apply each source-task skill to controlled variants without further evolution. Input-representation changes (P1) alter names, column arrangements, or schemas; task-instance changes (P2) replace the data or instance structure; coupled-requirement changes (P3) alter or add requirements that must be satisfied together. The corresponding decision demands are to remap the input, recompute a solution for a replacement instance, and accommodate changed correctness requirements. A frozen skill must support these adaptations through guidance already present in the artifact.

The complete record contains 222 variants and 886 condition-level results across no skill and the three methods. Table 1 summarizes historical and recovery evaluations for 222 SkillEvolver variants and 221 each for SPARK and CoEvoSkills, pairing each skill condition with the no-skill result on the same variant.

Source-task improvement does not imply useful generalization. All three methods improve mean reward on the source tasks in both runs, but their variant results diverge. SkillEvolver retains a positive mean gain of 11.94 points, SPARK is close to neutral at $+0.65$, and CoEvoSkills falls below the no-skill baseline at -5.01. SkillEvolver also improves more variants than it regresses (47 versus 20), whereas SPARK and CoEvoSkills show the reverse. The improvement and regression counts clarify the near-zero SPARK mean: it combines changes in opposite directions, rather than uniformly negligible effects. SkillEvolver shows both a larger positive mean and more frequent improvement than regression.

Replacing the instance produces the largest divergence. SkillEvolver remains positive under all three changes, with its largest gain on P2 ($+14.56$). P2 is also where SPARK and CoEvoSkills decline most (-5.23 and -9.51). SPARK retains positive averages under representation and requirement changes, but loses its advantage when the underlying instance changes. The P2 improvement–regression counts reinforce this divergence: 15 versus six for SkillEvolver, eight versus 14 for SPARK, and six versus 13 for CoEvoSkills. Replacing the instance changes both the average utility and the prevalence of helpful versus harmful changes. SkillEvolver's positive gains across P1–P3 show the broadest observed persistence of utility, while CoEvoSkills' negative means show that source-task guidance can become counterproductive after the task changes.

Method	Evaluation	N	No skill	Skill	Δ	I	R	T	G	L
SkillEvolver	Source run 1	81	30.20	55.81	+25.61	27	6	48	30.91	5.30
	Source run 2	81	37.32	62.34	+25.02	26	3	52	28.32	3.30
	Variants: P1	77	42.04	53.89	+11.85	15	7	55	17.26	5.41
	Variants: P2	68	45.00	59.56	+14.56	15	6	47	22.06	7.50
	Variants: P3	77	38.94	48.65	+9.72	17	7	53	17.73	8.01
SPARK	Source run 1	84	17.82	19.66	+1.84	9	8	67	9.38	7.54
	Source run 2	84	14.37	20.81	+6.45	13	5	66	12.40	5.95
	Variants: P1	77	42.04	44.05	+2.00	12	11	54	13.43	11.43
	Variants: P2	68	45.00	39.77	−5.23	8	14	46	11.76	16.99
	Variants: P3	76	38.13	42.66	+4.53	12	12	52	15.79	11.26
CoEvoSkills	Source run 1	85	42.21	52.44	+10.24	17	9	59	17.14	6.90
	Source run 2	85	23.54	33.31	+9.76	17	10	58	18.26	8.50
	Variants: P1	77	42.04	38.46	−3.59	9	14	54	11.04	14.63
	Variants: P2	68	45.00	35.49	−9.51	6	13	49	8.09	17.60
	Variants: P3	76	38.13	35.71	−2.42	11	14	51	13.50	15.92

Baseline-conditioned variant outcomes

Method	Baseline zero	Zero to positive	Zero to full	Baseline full	Full to lower
SkillEvolver	126	42	40	86	16
SPARK	126	30	30	85	29
CoEvoSkills	126	25	23	85	32

Table 1. Completed-skill utility on source tasks and controlled variants. Rewards, paired gains Δ, positive contributions G, and negative contributions L use a 0–100 scale, with $\Delta = G - L$ before rounding. I/R/T counts improved, regressed, and tied tasks. P1: input representation; P2: task instance; P3: coupled requirements. Variant rows describe the complete historical and recovery record, pairing each skill result with its same-variant no-skill baseline. The lower panel counts transitions from zero or full baseline reward; each method also has ten intermediate-baseline variants.

4. ANALYSIS

We distinguish three contributors to mean utility: agreement on which tasks improve across runs, the balance between positive and negative changes, and whether the skill helps or harms execution at different no-skill reward levels.

4.1. Recurring Means Do Not Imply the Same Tasks Improve

Matching task identities across the two source-task runs reveals how much of the positive mean comes from recurring improvements. SkillEvolver improves 27 tasks in run 1 and 26 in run 2, but only 14 improve in both. The corresponding overlaps are three for SPARK and seven for CoEvoSkills. Across tasks that improve at least once, these counts are 14 of 39, three of 19, and seven of 27. Thus, the nearly unchanged mean gains of SkillEvolver and CoEvoSkills coexist with changes in which tasks benefit.

The transitions clarify this variation. Of SkillEvolver's 27 first-run improvements, 14 remain improvements, 12 become ties, and one becomes a regression. SPARK's nine first-run improvements become three improvements, four ties, and two regressions. CoEvoSkills' 17 become seven improvements, nine ties, and one regression. Losing an observed advantage most often means becoming tied with the no-skill execution, rather than reversing the sign. Conversely, ten, seven, and ten tasks move from a first-run tie to a second-run improvement for SkillEvolver, SPARK, and CoEvoSkills, respectively.

CoEvoSkills also regresses on five of the same tasks in both runs; neither other method has a task that regresses twice. This identifies a recurring source-task cost within a positive overall mean. The two runs therefore supply two distinct observations: each method has positive aggregate utility twice, while the task identities supporting that utility are only partly shared. These are comparisons of paired outcomes across runs; the shared object is the task, with each run using its recorded skill condition and execution.

4.2. Benefits and Losses Can Nearly Cancel

To distinguish small effects from cancellation, let $d_i = r_i^{\text{skill}} - r_i^{\text{no-skill}}$. We separate the variant mean into positive and negative contributions:

$$G = \frac{100}{N} \sum_i \max(d_i, 0), \quad L = \frac{100}{N} \sum_i \max(-d_i, 0). \tag{5}$$

The paired gain is $\Delta = G - L$. Both terms use all evaluated variants as their denominator, so they account for the frequency and magnitude of score changes together.

SkillEvolver contributes 18.89 points of improvement against 6.95 points of loss, leaving the observed +11.94 gain. SPARK contributes 13.73 positive points and 13.08 negative points. Its +0.65 mean therefore masks substantial movement in both directions. CoEvoSkills contributes 10.98 positive points, but its 15.98-point loss outweighs them. Its negative average includes useful changes on some variants alongside a larger cost elsewhere.

The conditional magnitudes further distinguish these patterns. Among improved variants, the average increase is 89.23 points for SkillEvolver, 94.82 for SPARK, and 93.30 for CoEvoSkills. Among regressed variants, the average loss is 77.16, 78.15, and 86.16 points. All three methods can produce large gains where they help. Their overall separation comes from how often those gains occur and how

much is lost on other variants, rather than SkillEvolver alone producing large individual improvements. Its 47 improvements and 20 regressions give positive changes more weight; the balance reverses for SPARK (32 versus 37) and CoEvoSkills (26 versus 41).

4.3. Helping Zero-Reward Tasks and Preserving Full-Reward Tasks

Conditioning on the no-skill reward gives the changes a concrete interpretation. Each method's variant set contains 126 zero-reward no-skill executions. SkillEvolver raises 42 of them to positive reward, including 40 to full reward. SPARK raises 30 to full reward, and CoEvoSkills raises 25 to positive reward, including 23 to full reward. The respective rates of moving beyond zero are 33.3%, 23.8%, and 19.8%. SkillEvolver's advantage thus includes more frequent gains on variants where the executor receives no verifier credit without the skill.

The other endpoint measures losses on variants with a full-reward no-skill execution. Among SkillEvolver's 86 full-reward no-skill variants, 16 fall below full reward, including 15 that fall to zero. SPARK lowers 29 of 85, including 27 to zero; CoEvoSkills lowers 32 of 85, all to zero. The rates of losing full reward are 18.6%, 34.1%, and 37.6%. Each method also has ten intermediate-baseline variants, which remain included in the aggregate results but are outside these endpoint counts.

These two directions explain why counting newly successful tasks alone is insufficient. SPARK and CoEvoSkills both turn many zero-reward cases into full-reward outcomes, yet lose full reward on almost as many or more baseline successes. SkillEvolver combines the highest rate of gains from zero with the lowest rate of losses from full reward. Its broader utility is expressed in both acquiring a benefit and preserving outcomes that the executor already achieves.

4.4. Useful Guidance Must Survive a Change in Demand

Source-task utility establishes whether a completed skill helps on the task from which it was constructed. Frozen-skill variants test whether its guidance remains useful when original details no longer apply. Their three demands—remapping inputs, recomputing instance-dependent choices, and accommodating coupled requirements—probe different scopes of useful behavior.

This distinction is central to the observed method profiles. SPARK retains positive means under P1 and P3 but becomes negative under P2. Its source-task benefit therefore does not extend uniformly to replacement instances. CoEvoSkills has positive source-task means but negative means under every variant type. SkillEvolver combines positive means with more improvements than regressions in each type. The latter profile is the broadest evidence of useful generalization in these evaluations: the frozen guidance remains beneficial across different demands on the executor.

The published construction procedures suggest where these demands enter skill learning. SkillEvolver contrasts successful and failed deployments and audits reliance on training-instance constants [1]; SPARK distills environment-verified execution trajectories [2]; CoEvoSkills uses a co-evolving surrogate verifier to supply detailed revision feedback [3]. These mechanisms draw on different evidence when retaining guidance. Our behavioral results make abstraction from the source instance a concrete target for explaining their differences, especially the divergence under P2. Testing that explanation would require changing the construction feedback while holding the evaluation fixed.

4.5. Learning a Useful Skill Includes Avoiding Interference

The endpoint analysis adds a second requirement to generalization: a skill should help when the executor struggles without displacing decisions it already makes correctly. The no-skill condition makes both directions observable. Moving from zero to positive reward indicates additional useful behavior in the recorded execution. Falling from full reward identifies a lower-scoring skill execution paired with a full-reward no-skill execution. Reporting only the former would conceal a substantial part of the effect.

The SPARK and CoEvoSkills results show why this distinction changes the interpretation of learning. Both produce many large improvements, so their weak or negative variant means cannot be explained by an absence of helpful skills. Instead, their gains are offset by losses elsewhere. SkillEvolver's stronger net utility combines more gains from zero with fewer disruptions of full-reward outcomes. The diagnostic target is therefore the distribution of useful guidance: where it adds value, where it remains unnecessary, and where it interferes with execution.

5. RELATED WORK

Learning persistent skills from experience. SkillEvolver, SPARK, and CoEvoSkills make experience persistent through different construction loops: deployment-grounded revision, verified-trajectory distillation, and co-evolutionary verification, respectively [1–3]. These methods motivate our focus on the artifact delivered to a later executor. We study how that artifact's usefulness changes after its source task is varied, and analyze the task-level benefits and losses that produce its overall utility.

Evaluating skill utility and use. SkillsBench evaluates skill utility across diverse tasks, while SWE-Skills-Bench studies skill use in software engineering [4, 5]. SkillLearnBench evaluates continual skill generation on real-world tasks [6]. From Raw Experience to Skill Consumption separates experience generation, skill extraction, and downstream consumption [7]. Our study connects these evaluation levels to the identification of skill learning: repeated outcomes characterize completed-skill utility, task changes test its persistence, and score-transition analyses distinguish added benefits from losses of existing success. The unit of interpretation is the persistent artifact in use.

6. CONCLUSION

Our diagnosis examines what score gains from self-evolving skills represent beyond the source task. All three methods improve source-task means in two runs, but the individual tasks benefiting from them only partly recur. Frozen-skill variants expose different scopes of generalization, with SkillEvolver sustaining positive utility across representation, instance, and requirement changes. Decomposing the results shows that all three methods can deliver large individual gains; their net utility depends on the frequency of those gains and the losses incurred elsewhere. SkillEvolver combines more frequent gains from zero reward with fewer disruptions of full-reward executions. These findings connect the learning question to observable behavior: reusable guidance should add useful capability, retain its value when the task changes, and preserve success that the executor already achieves.

7. REFERENCES

[1] Genrui Zhang, Erle Zhu, Jinfeng Zhou, Caiyan Jia, and Hongning Wang, "SkillEvolver: Skill Learning as a Meta-Skill," 2026.

[2] Yang Zhou, Zihan Dong, Zhenting Wang, Can Jin, Shiyu Zhao, Bangwei Guo, Difei Gu, Linjun Zhang, Mu Zhou, and Dimitris N. Metaxas, "Evidence Over Plans: Online Trajectory Verification for Skill Distillation," 2026.

[3] Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, Xue Liu, Xiaoxiao Li, and Philip S. Yu, "CoEvoSkills: Self-Evolving Agent Skills via Co-Evolutionary Verification," 2026.

[4] Xiangyi Li, Yimin Liu, Wenbo Chen, Bingran You, Zonglin Di, Yifeng He, Shenghan Zheng, Kyoung Whan Choe, Jiankai Sun, Shuyi Wang, Chujun Tao, Binxu Li, Xuandong Zhao, Hejia Geng, Xiaojun Wu, Junwei Zhou, Xiaokun Chen, Hanwen Xing, Yubo Li, Qunhong Zeng, Di Wang, Yuanli Wang, Roey Ben Chaim, Penghao Jiang, Haotian Shen, Luyang Kong, Xinyi Liu, Runhui Wang, Xuanqing Liu, Jiachen Li, Xin Lan, Yueqian Lin, Wengao Ye, Junwei He, Songlin Li, Yue Zhang, Yipeng Gao, Yijiang Li, Ze Ma, Liqiang Jing, Tianyu Wang, Kaixin Li, Yiqi Xue, Haoran Lyu, Yizhuo He, Yuchen Tian, Shutong Wu, Bowei Wang, Yixuan Gao, Bo Chen, Litong Liu, Sikai Cheng, Jiajun Bao, Shuaicheng Tong, Shuwen Xu, Terry Yue Zhuo, Tinghan Ye, Qi Qi, Miao Li, Longtai Liao, Zelin Tan, Chang Shi, Xilin Tang, Srinath Tankasala, Boqin Yuan, Yaoyao Qian, Jianhong Tu, Chenguang Wang, Yizhou Sun, Wei Wang, Aaron Taylor, Ziyue Yang, Changkun Guan, Zhikang Dong, Xinyu Zhang, Steven Dillmann, Han chung Lee, and Dawn Song, "SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks," 2026.

[5] Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu, "SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering?," 2026.

[6] Shanshan Zhong, Yi Lu, Jingjie Ning, Yibing Wan, Lihan Feng, Yuyi Ao, Leonardo F. R. Ribeiro, Markus Dreyer, Sean Ammirati, and Chenyan Xiong, "SkillLearnBench: Benchmarking Continual Learning Methods for Agent Skill Generation on Real-World Tasks," 2026.

[7] Zisu Huang, Jingwen Xu, Yifan Yang, Ziyang Gong, Qihao Yang, Muzhao Tian, Xiaohua Wang, Changze Lv, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Xue Yang, Dongdong Chen, Xiaoqing Zheng, and Chong Luo, "From Raw Experience to Skill Consumption: A Systematic Study of Model-Generated Agent Skills," 2026.